

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 2

Student Name: A.Rohit

Roll No.: 324126510006

Date: 30-08-2026

1. TITLE

Design and Implementation of Interactive UI Layouts using CSS3 and Canvas

2. AIM

To style a static HTML5 webpage using CSS3 selectors, box models, Flexbox, CSS Grids, transitions, and to draw interactive 2D graphics utilizing the HTML5 Canvas API.

3. OBJECTIVE

The objectives of this experiment are:

- To apply internal CSS rules with class and ID selectors.
- To implement complex background gradients and text shadows.
- To design responsive flexbox and grid dashboard layouts.
- To create interactive button hover transitions and transformations.
- To programmatically render shapes, lines, and text onto an HTML5 Canvas.

4. THEORY

Cascading Style Sheets (CSS3) is the styling language used to describe the presentation of a document written in HTML. CSS3 introduces advanced features like rounded corners, shadows, gradients, transitions or animations, and flexible box/grid layouts.

Flexbox provides a more efficient way to lay out, align, and distribute space among items in a container, even when their size is unknown or dynamic.

The HTML5 <canvas> element is used to draw graphics, on the fly, via JavaScript. The canvas is simply a container for graphics, while a script is required to actually draw the graphics (paths, boxes, circles, text, and adding images).

Application Flow:

HTML Document -> CSS Styling (Colors, Fonts, Layout) -> DOM Rendering -> Canvas API Initialization -> 2D Graphics Drawing

5. ALGORITHM / PROCEDURE

1. Create a semantic HTML structure with navigation, header, and content sections.
2. Apply linear-gradient backgrounds to the document body.
3. Target elements using CSS IDs (#main-nav) and classes (.feature-card) to apply paddings, margins, and borders.
4. Define display: flex; on a container to align its children horizontally with gap spacing.
5. Implement pseudo-classes like :hover on buttons to trigger CSS transforms (scale) and background color transitions.
6. Embed a <canvas> element with specified width and height.
7. Write an embedded JavaScript block to access the canvas 2D context.
8. Use context methods (fillRect, arc, moveTo, lineTo, fillText) to draw geometric shapes and names.
9. Test the layout responsiveness and hover animations in the browser.

6. PROGRAM / SOURCE CODE

Source File: task2.html

File not found

7. CODE EXPLANATION

Code / Syntax	Explanation
background: linear-gradient(...)	Creates a smooth transition between two or more specified colors.
box-shadow: 0px 4px 10px rgba(...)	Attaches one or more drop-shadows to the element's bounding box.
display: flex;	Initiates a flexbox formatting context for its children.
transition: transform 0.3s ease;	Provides a smooth animation effect when CSS properties change (e.g., on hover).
canvas.getContext('2d')	Returns a two-dimensional drawing context on the canvas.

<code>ctx.arc(x, y, r, 0, 2*Math.PI)</code>	Creates a circular arc/curve path.
<code>ctx.fillText(text, x, y)</code>	Draws filled text on the canvas at the specified coordinates.

8. TEST CASES AND EXECUTION DATA

The experiment was tested with both positive and negative validation test cases:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Render CSS Flexbox	Dashboard cards align horizontally with space-around	Cards successfully rendered in a flex row layout	Pass
TC-02	CSS Hover Transition	Button scales up and changes color on mouse hover	Smooth transition observed on hover state	Pass
TC-03	Canvas Context Rendering	Draw orange rectangle, green circle, and blue line	Shapes painted to canvas successfully	Pass

9. OUTPUT

10. TEST CASES SCREENSHOTS

11. RESULT

Thus, the experiment was successfully implemented and verified across both positive functional and negative validation test cases.